Library Management System - STIQ Library

COP4710 Database Management Spring 2026

Group 42 Members: Isabella Correa, Quincy Joseph, Sofia Murillo, Tiffany Ortiz

1. Project Overview

This project is a library management database made to help organize and manage important library information. The database is designed to manage books, members, returns, rentals, and fines all in one place. Each group member worked on different parts of the system, including views, triggers, queries, dashboards, and constraints. The system also supports basic library staff tasks. Staff can add new books, edit existing book records, add new members, and update member information when needed. They can also check out books for members and process returns through the system. This makes the database useful for handling normal day to day library activity, not just for storing data.

We built a dashboard that gives a quick overview of what's going on in the library. It shows things like total books and copies, active members, active rentals, overdue books, and total fines. There are buttons to show the main queries, so instead of writing SQL every time, you can just click and see our tables for overdue books, popular books, rental history, and unpaid fines. It makes everything way easier to access.

We used triggers to automate important things. For example, one trigger calculates a fine automatically if a book is returned late, another creates a fine if a book is damaged or lost, another updates the number of available copies when a book is rented or returned, and another blocks suspended members from renting. This helps avoid mistakes and makes the systems run smoother.

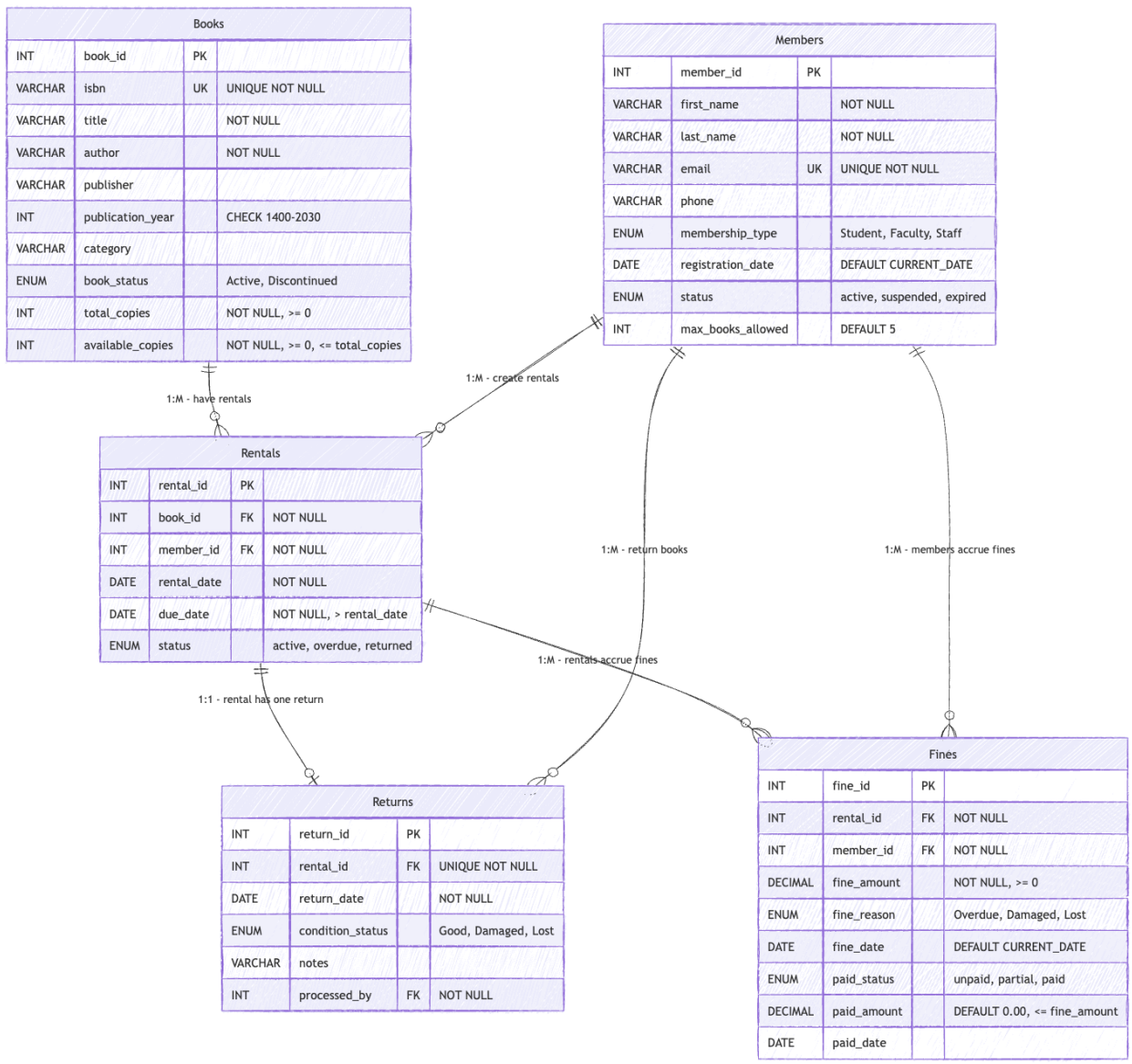
We also created 10 main queries that give useful information about the library. These include things like books that were never borrowed, rental history, recent rentals, monthly activity, and fines. We picked these because they're the kind of info someone running a library would actually want to see. We also added constraints to make sure the data entered is valid, so that it prevents invalid values, missing info, duplicates, or wrong dates. Overall, the goal is to make managing the library easier and keep everything organized and accurate.

Our database has 36 books, 36 members (4 staff and 32 students /faculty), 60 rentals (35 returned, 11 overdue, 14 active), 35 returns, and 15 fines.

2. Schema Summary

The schema represents a fully structured library management system where each table in the schema does a specific role in tracking books, members, rentals, returns, and fines that are connected by 1:1 and 1:many relationships while also enforcing data integrity rules.

The Books table stores all book details and inventory counts, and each book can have many rentals, creating a 1:many relationship with the rentals table. The Members table holds member information and also connects to the Rentals table through a 1:many relationship, since each member can check out multiple books. The Rentals table links both books and members while implementing valid rental dates and statuses. Each rental can have one return, which implements the 1:1 relationship with the Returns table, which keeps track of the return date, condition, and staff member who processed it. The Fines table keeps track of penalties for overdue, damaged, or lost books and also connects to both Members and Rentals through a 1:many relationship, since one member or rental can get multiple fines.



3. Advanced Features

How queries are routed in app.py:

- All 10 main queries and 5 simple statistic queries are found in the dashboard. The 5 simple stat queries are at the top of the dashboard and are routed through queries.py found in the backend. They are listed in a dictionary called DASHBOARD_SQL. The app.py routes the 5 dashboard queries using the command query_db() using the route @app.route('/api/stats').
- The 10 main queries are also found in queries.py listed in a dictionary called QUERIES_SQL. The queries are each displayed using the route @app.route('/api/queries/<int:query_id>'). Using the route it gets each queries id using sql = QUERIES_SQL.get(query_id) so if we are trying to look at query 4 http://localhost:5002/api/queries/4 calls api_query(4).
- For the first 3 views, they are connected to the first 3 queries in our queries.py file, where we list the column order for the frontend display. In the app.py file we set app.json.sort_keys = False, it defaults as true, and this causes jsonify to order all columns alphabeticaly. We decided to dictate the columns order in the queries.py file to make sure everything is displayed the way we want.
- The next 7 queries are displayed after the views and they give more stats on our database.

Table routing and Search functionality:

- Search functions are used for the Books and Members page tables. Search is done by taking the users input and appending them to the query using LIKE statements, to check if any of the rows match the users input.
- The Booke table app routing imports query View #2 from views.sql to show all data for the books table, it also updates all the books availability by checking the available copies. Books has a search function for title, author, or ISBN, this is done by appending the users input and checking if it matches any of the parameters.
- The Members table we display members using a query to display all info for members. It has a search function like the books table where the user can search a member by using first name, last name, or email.
- Rentals shows all active rentals, and the user can checkout a book and it will update the table using member name, book, rental date and due date.
- Returns show all return history for the library, the user can return a book.
- Fines show all fine history for the database, the user does not input their own fines, they are automatically calculated using a trigger.

Create, update, and delete data in db:

- For CRUD, we add user input and insert them into queries and it is executed by the db using execute_db(). The user can add new books, members, rentals, and returns to the database. We use the POST sql command and an Insert query to add the users input into the query so the db can execute it.
- The user can edit existing books data and members data by using the PUT command. We append the updated user input into the Update query and it directly changes the data in our mysql db.
- The user can also delete books by using the DELETE command, it executes a Delete query and uses the book_id of the book the user is trying to delete.

How views work and connect;

- Views are saved sql queries that we treat just like a table. Views are defined once, and then everytime we write a SELECT * FROM view_name, mysql will run the query and give us new results. This is valuable because as things change in our database, data changes, and we need updated views of tables. Instead of storing the data in a table separately, it acts just like a shortcut to what we want to see without having to write a longer query every time. Instead we just use the Select line calling that views name and it updates. So as data is added and we want to see an update, for example for view #3, everytime someone presses the View 3: Most Popular Books button in the frontend it will update the table to show the most up to date list.
- There are 3 views in views.sql in the backend folder. They are queries that connect to the first 3 queries in queries.py. The views are defined in views.sql and then in the column ordering is defined in queries.py, so that the frontend shows the columns the way we want to be displayed. The full flow for how this works is the frontend calls the api route first, then flask runs the the Select statement calling the views name, then mysql executes the full query found in views.sql, and then flasks receives and send the results back as JSON that we can then display.
- View 1 joins the Rentals, Books, and Members tables to show all books that are currently overdue. It filters the results to only include rentals that are still active or already marked overdue and are past their due date. It also calculates how many days the book is late using DATEDIFF with the current date, and from that it estimates a fine at $1 per day. Since it uses the current date every time the viewis run, the days overdue and estimated fine are always up to date.
- View 2 pulls information from Books table and uses a CASE statement which labels each book as either ‘Available’, ‘Low Availability’, ‘Discontinued’ or ‘Unavailable’ based on the value in available_copies. Also calculates the value of

copies_on_loan from total_copies - available-copies and uses a subquery to find the the value of next_available_date.

- View 3 shows the top 10 most popular books in our library. It uses a JOIN between Books and Rentals, and then uses a variable called times_borrowed that stores COUNT(rental_id), to count the total number of rentals each book has in our db history. It uses an INNER JOIN so that means books that have never been rented will never show up. We only want 10 books to display in the table for the top 10 so we add a LIMIT 10 to the query. If we ever want to make a new and longer list, all we have to do is increase this limit.

<u>How triggers work and connect:</u>

- Triggers are sql code that mysql automatically can execute whenever an event occurs to trigger it. They arent done manually, they are done automatically as a response to us adding or updating or deleting any data. We have 1 before trigger, a before trigger happens before a row is written into our tables, so if there is an error somewhere, we can block that data from being added or edited. We have 3 after triggers, they are run after a row has been saved to the db, it reacts to the insert/update/delete of data, and it can edit columns of data. It can do calculations, and update any counts we ask it to for our db.
- There are 4 triggers found in triggers.sql in the backend folder. They are sql queries that run inside of mysql. Mysql automatically executes these triggers when a specific event triggers the update.
- Trigger 1 is to auto calculate fines when a book is returned. This trigger occurs after an INSERT on the Returns table. So when the user returns a book, it will add that return to the table, and mysql will automatically look up that rentals due date, and then calculates how many days late the return is. If its late, then it inserts a fine into the Fines table with all the return data in it. This trigger marks the rental data as returned in the Rental table, and it also updates the books available_copies by 1. This trigger is used every single time a new book is returned.
- Trigger 2 runs after a return is added to the Returns table. If the book's condition is marked as damaged, it creates a $25 fine, and if its marked as lost, it creates a $50 fine. This works along with the overdue fine trigger, so if a book is returned late and damaged, it can end up with two fines.
- Trigger 3 functions to decrement the available copies section by one each time someone checks out a book, which happens in the rentals table. This trigger ensures that a book that is currently rented is not available to users, preventing mistakes.

- Trigger 4 happens before a rental is inserted to make sure that the member who is renting a book is cleared to rent one. This means that any member who is suspended can't rent a book and a rental row won't be created.

Api routes

- Each table/query is routed to their own pages. Here are all the routes used and what they do and display in the frontend.

GET /api/books	Book table and search
POST /api/books	Add new book
PUT /api/books/<id>	Edit existig book
GET /api/members	Members table and search
POST /api/members	Add new member
PUT /api/members/<id>	Edit existing member
GET /api/rentals	Rentals table
POST /api/rentals/	Checkout book
GET /api/returns	Returns table
POST /api/returns	Process a return
GET /api/fines	Fines table
GET /api/stats	Dashboard 5 stats
GET /api/queries/<id>	Queries 1-10

4. Individual Contribution Statement

Isabella Correa

Tables worked on:

I worked on the Books and Fines tables found in schema.sql. 36 books and 15 fines were added to the data.sql. There are multiple copies of each book, and we want multiple rentals per book, so we have a lower amount of book data compared to rental data.

Books table uses primary key book_id and attributes isbn, title, author, publisher, publication_year, category, book_status, total_copies, available_copies.

Fines tables has primary key fine_id, foreign keys rental_id and member_id and attributes fine_amount, fine_reason, fine_date, paid_status, paid_amount, paid_date.

Queries written:

VIEW #3: Most Popular Books connects to QUERY 3: Most Popular Books:

- View #3 can be found in views.sql it is a view to show the libraries top 10 most popular books, it counts the number of times a book has been checked out, it orders then my the count held in times_borrowed descending, and limits the number of rows to display only 10. This view used to be a regular query in queries.py, we connected this view to that query #3. We select the columns we want displayed for the view in queries.py.

TRIGGER #4: Prevent a Rental If Suspended

- Trigger #4 is found in triggers.sql it prevents rentals if someone tries to log a new rental for a suspended member. It gets an error message to the console and to the frontend that the members account is suspended and that they cannot rent a book.

QUERY #4: Books Never Borrowed

- Query #4 is found in queries.sql, it is used to find a list of books that have never been borrowed by a member in the library. It uses a left join and checks the rental table for rental_id = NULL for each book.

QUERY #10: Unpaid / partial fines

- Query #10 is found in queries.sql, it lists all the unpaid and partial fines in the database, it checks the fines paid status to include unpaid and partial, and orders the table by the fine_date.

Constraints:

Attempt to insert a fine with an invalid paid_status value.

- I attempted to insert a new fine and set paid_status to ‘complete’ which is an invalid value and does not match any of the ENUM options in the fines schema.

Attempt to delete a book that is still referenced by rentals.

- I try to delete the book with book_id, but it has an open rental, so it does not allow it to delete from the db.

Attempt to insert an active member with a negative max_books_allowed value.

- This tests the max_books_allowed constraint on the members schema, I try to set it to -5 which is an invalid negative value, it blocks the insert.

Attempt to insert a rental where due_date is before rental_date

- This tests the constraint on the dates for the rental schema. I try to set the due date 7 days before the rental date, and this blocks the insert, the due date must always be after.

<u>Backend features implemented</u>

I worked on the app.py backend, routing the tables to their specific pages. I worked on connecting views.sql and triggers.sql to the database through app.py. All queries in queries.py are displayed using routes that check for each query id. I implemented routes for the 5 schema tables, the 5 dashboard queries, and the 10 main queries. The 3 view queries are connected to queries.py which holds a dictionary of all queries, and then app.py routes those queries based on their id. The 4 triggers are used to update the database whenever a specific event occurs. They are defined in triggers.sql and mysql runs them automatically. I worked on search functionality using parameters for Books and Members so it can take user input and append them to the queries. I worked on CRUD commands for Books, Members, Rentals, and Returns. Books uses POST and PUT to add new books and update existing books, Members use POST AND PUT to add new members and update existing members, Rentals uses POST to checkout new books, Returns uses POST to process a return.

Sofia Murillo

Tables worked on:

I was responsible for the rentals table which uses rental_id as its primary key. The attributes in this table are book_id, member_id, rental_date, due_date, status.

Queries worked on:

TRIGGER 2: Auto create a damage fine

- Trigger #2 is in triggers.sql. This trigger creates fines automatically after a book is returned damaged or is lost. The amount fined will differ depending on whether the book is damaged or lost.

QUERY #6: Rental history with return details

- Query #6 is located under queries.sql and its function is to show the rental history of a book as well as the status details of the book's conditions. This will affect any fines regarding the book. This query will check if the book has been returned on time, late, or not returned at all.

QUERY #7: Recent Rentals (last 7 days)

- Query #7 can be found in queries.sql. This query uses two inner joins to find which books have been rented in the past seven days.

-- Dashboard Query 1: book titles and total copy count

- This query provides the titles of books in the system and how many copies we have.

-- Dashboard Query 5: total outstanding fines for the entire library.

- Dashboard query five is used to show an overview of all the fines that haven't been pai in the library.

Constraints:

Attempt to insert a return for a rental_id that already exists in Returns.

- This constraint prevents any duplicate returns.

Attempt to insert a book with a NULL title.

- Prevents a book with no title (NULL value) from being inserted into the table.

Attempt to insert a fine with a NULL fine_amount.

- Prevents fines from being added with null amounts.

Attempt to insert a rental with an invalid status value.

- Every rental needs a status defined within the parameters, so this prevents an invalid status from being inserted.

Tiffany Ortiz

Table worked on:

I worked on the Members table found in schema.sql. This table stores information about each library member.

The Members table uses primary key member_id and includes attributes first_name, last_name, email, phone, membership_type, registration_date, status, and max_books_allowed. The table also includes a check constraint to make sure max_books_allowed is greater than 0, and that each email is unique so no duplicates happen.

VIEW 1 Overdue Books connects to (QUERY 1: All Currently Overdue Books)

- VIEW 1 can be found in views.sql and shows all the overdue books in the library. It uses data from the rentals and books tables to figure out which books are past their due date. This view is connected to Query #1, which pulls the overdue data to be used in the system. I just picked the columns we needed in queries.py to show the important info.

TRIGGER 1: Auto Calculate a Fine On Return

- Trigger #1 is in triggers.sql and it automatically adds a fine when a book is returned late. When a return gets added to the Returns table, it checks if the book is overdue and then creates a fine in the Fines table. This makes the process automatic so we don't have to do it manually and keeps everything consistent.

QUERY 5: Total Fines by Member

- Query #5 is in queries.sql and it finds the total fines for each member. It groups everything by member_id and adds up the fine_amount. This makes it easy to see how much each member owes overall.

-- Dashboard Query 2: members with active status

- This query shows all members who are marked as active. It filters the Members table using the status column so only active users show up. This makes it easy to see which members are currently valid.

-- Dashboard Query 4: all open rentals, counts overdue rentals as well.

- This query shows all rentals that are still open, including the ones that are overdue. It checks the rental status and only includes records that haven't been returned yet. This helps keep track of what's still out and which ones are late.

Constraints:

Attempt to insert a return for a rental_id that already exists in Returns.

- Prevents inserting a return with a rental_id that already exists in the Returns table, so duplicate returns don't happen.

Attempt to insert a book with a NULL title.

- Prevents inserting a book with a NULL title, so every book has a name.

Attempt to insert a fine with a NULL fine_amount.

- Prevents inserting a fine with a NULL fine_amount, so all fines have a value.

Attempt to insert a rental with an invalid status value.

- Prevents inserting a rental with an invalid status, so only allowed status values are used.

Quincy Joseph

Tables worked on:

I worked on the Returns table found in schema.sql. This table records all book return actions, including the condition of the book and the staff member who processed the return. It uses a 1 to 1 relationship with the Rentals table, which makes each rental only being able to be returned one time.

The Returns table uses the primary key return_id, and includes attributes such as rental_id, return_date, condition_status, notes, and process_by. The table also utilizes constraints such as unique constraint which makes sure each rental_id appears only one time in Returns and a ENUM constraint which restricts the condition_status to 'Good, Damaged, or 'Lost'.

View 2: Book availability connects to (Query 2: Books by Availability Status)

- View 2 is found in views.sql and shows the availability of each book based on the number of available copies. It uses data from the Books table to list them as either 'Available', 'Limited', or 'Unavailable'.
- Query 2 uses the view to show book details such as the title, authour, category, total copies, available copies, and the next available date.
- The view and query makes ure that the system shows availability as it is happening in real time when a rental or return happens

Trigger 3: update_available_copies_on_rental

- This trigger is located in triggers.sql and automatically updates the number of coppies when a new rental is made
- When a record is put into Rentals, the trigger subtracts 1 from the book's available_copies, making sure the inventory is correct.
- Keeps Books table to be synced up with rental activity and stops over renting.

Query 8: Damaged or Lost Books Report

- This query is in queries.sql and lists all books returned in 'Damaged' or 'Lost' condition
- Joins Rentals, Returns, Books, Members, and Fines to give complete details about the incident, including the inforomation on the borrower and notes
- Helps staff track damaged or lost items and implements fines

Query 9: Monthly Rental Activity:

- This query is in queries.sql and shows rental activity by month and year
- Counts how may rentals happened in each month, showing trends
- Supports reporting and staffing decisions

Dashboard Query 3: Rentals Past Due

- Dashboard query counts all rentals thar are past due but still checked out
- Filters the Rentals by their status of either 'active' or 'overdue' and then compares the due_date to the current date

Constraints:
Attempt to insert a rental with an invalid status:

- I attempted to insert the status ‘lost’ for Rentals status but fails because lost is not a part of the Rental stats ENUM

Attempt to insert a fine with an invalid fine_reason

- I attempted to insert the value ‘complete’ for the Fines fine_reason but it fails because ‘complete’ is not a part of the ENUM

Check Constraint(Return days_overdue)
Attempt to insert a return with days_overdue = -5 and Attempt to update an existing return to a negative days_overdue

- I attempted to insert return with a value of -5 for its days_overdue and to update an existing return with a -2 value for days_overdue but both fail because the check constraint needs days_overdue to be greater than 0.

5. AI Use Disclosure and References

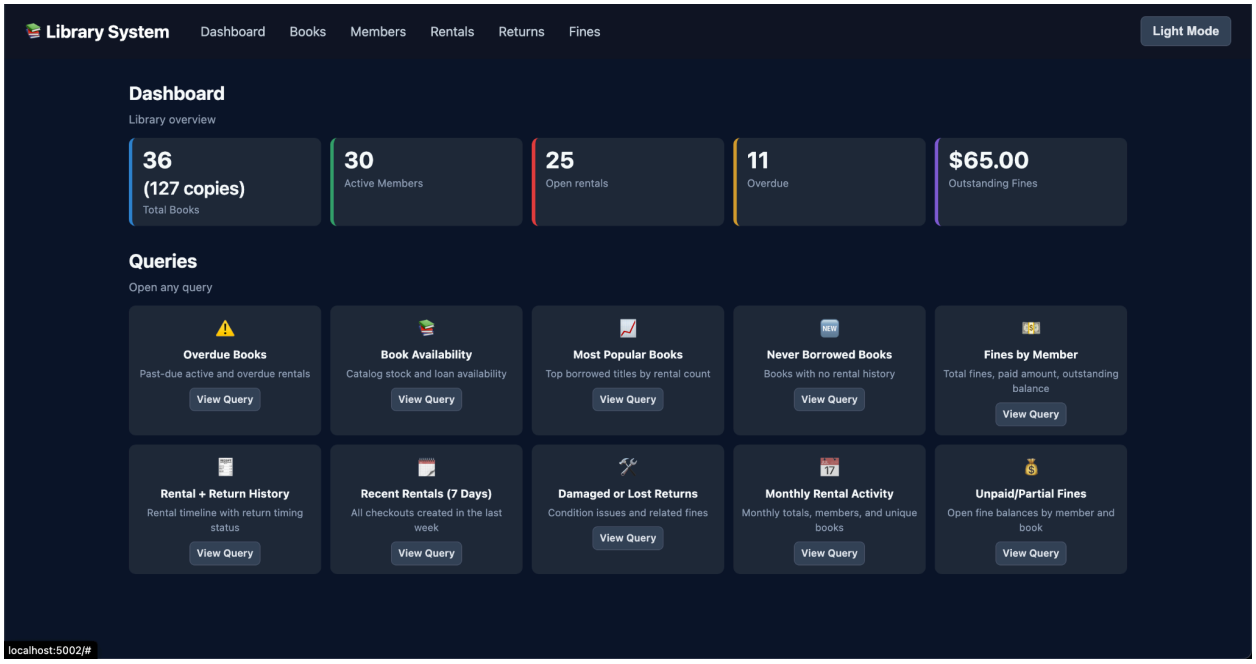
We used Claude for the frontend implementation files: index.html, api.js and app.js files and style.css.

References:

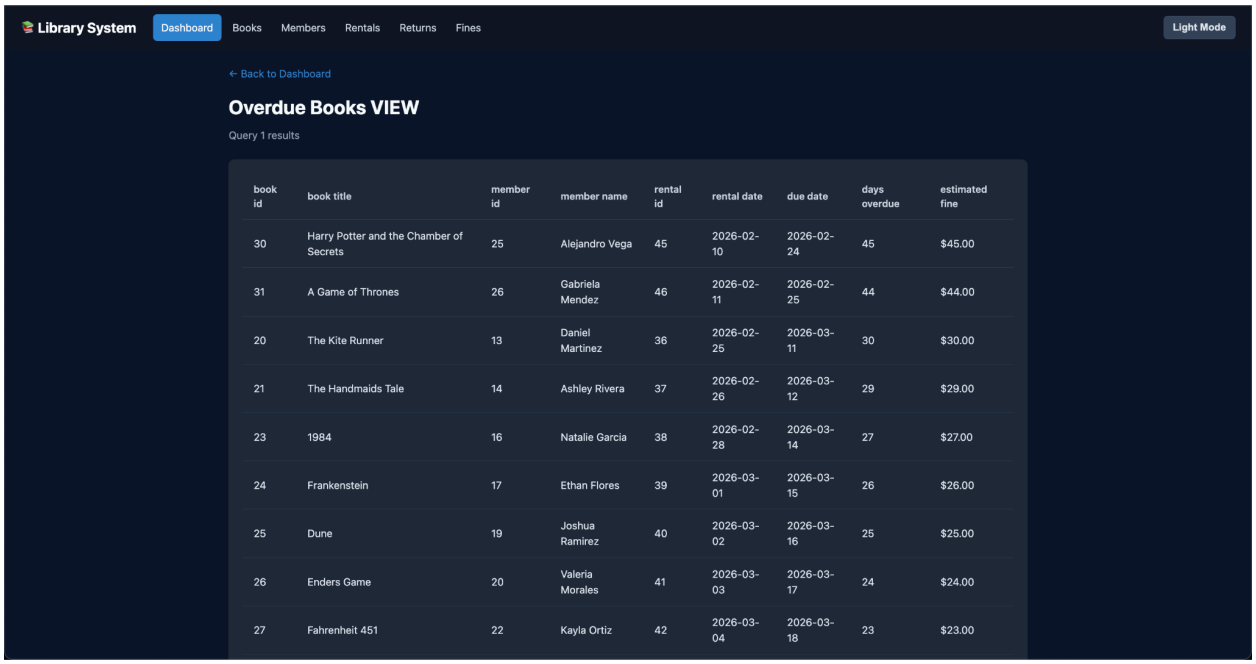
For Book data, we used https://isbnsearch.org/ to find book isbn information.

We used https://benefit.centerforfiction.org/200-books-that-shaped-200-years-of-literature and https://www.goodreads.com/list/show/18834.BBC_Top_200_Books to find some popular literature books, other 2000/2010’s books were based off our own knowledge and past read books that we wanted to include.

Screenshots:
Dashboard



Overdue Books View #1



book id	book title	member id	member name	rental id	rental date	due date	days overdue	estimated fine
30	Harry Potter and the Chamber of Secrets	25	Alejandro Vega	45	2026-02-10	2026-02-24	45	$45.00
31	A Game of Thrones	26	Gabriela Mendez	46	2026-02-11	2026-02-25	44	$44.00
20	The Kite Runner	13	Daniel Martinez	36	2026-02-25	2026-03-11	30	$30.00
21	The Handmaids Tale	14	Ashley Rivera	37	2026-02-26	2026-03-12	29	$29.00
23	1984	16	Natalie Garcia	38	2026-02-28	2026-03-14	27	$27.00
24	Frankenstein	17	Ethan Flores	39	2026-03-01	2026-03-15	26	$26.00
25	Dune	19	Joshua Ramirez	40	2026-03-02	2026-03-16	25	$25.00
26	Enders Game	20	Valeria Morales	41	2026-03-03	2026-03-17	24	$24.00
27	Fahrenheit 451	22	Kayla Ortiz	42	2026-03-04	2026-03-18	23	$23.00

Book Availability View #2

Library System | Dashboard | Books | Members | Rentals | Returns | Fines | Light Mode

← Back to Dashboard

Book Availability VIEW

Query 2 results

book id	title	author	publication year	publisher	category	total copies	available copies	copies on loan	availability status	next available date
18	Animal Farm	George Orwell	1996	Penguin Classics	Adult Fiction	4	4	0	Available	–
22	Gone Girl	Gillian Flynn	2012	Crown Publishing	Adult Fiction	1	0	1	Unavailable	2026-04-26
21	The Handmaids Tale	Margaret Atwood	1985	McClelland and Stewart	Adult Fiction	4	2	2	Low Stock	2026-04-30
20	The Kite Runner	Khaled Hosseini	2003	Riverhead Books	Adult Fiction	5	4	1	Available	2026-04-29
19	The Lord of the Flies	William Golding	2003	Penguin	Adult Fiction	4	3	1	Available	–
31	A Game of Thrones	George R.R. Martin	1996	Bantam Spectra	Fantasy	4	2	2	Low Stock	2026-04-30
30	Harry Potter and the Chamber of Secrets	J.K. Rowling	1999	Scholastic	Fantasy	5	3	2	Available	2026-04-29
29	Harry Potter and the	J.K. Rowling	1997	Scholastic	Fantasy	6	4	2	Available	2026-

Most Popular Books View #3

Library System | Dashboard | Books | Members | Rentals | Returns | Fines | Light Mode

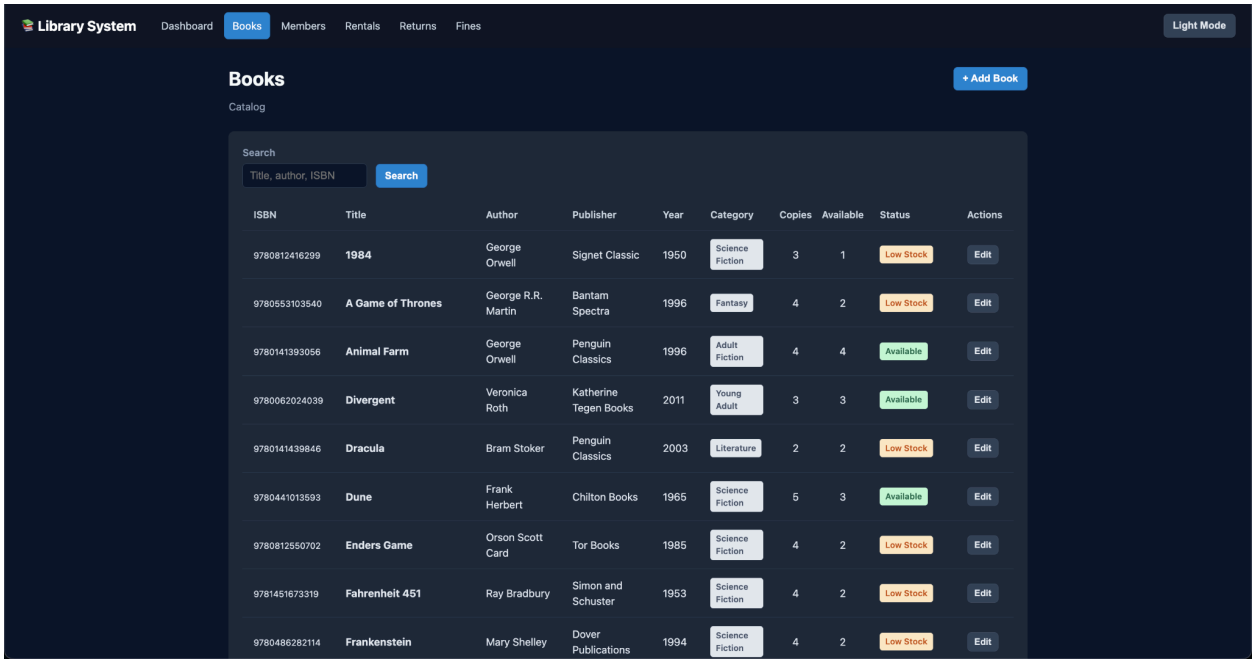
← Back to Dashboard

Most Popular Books VIEW

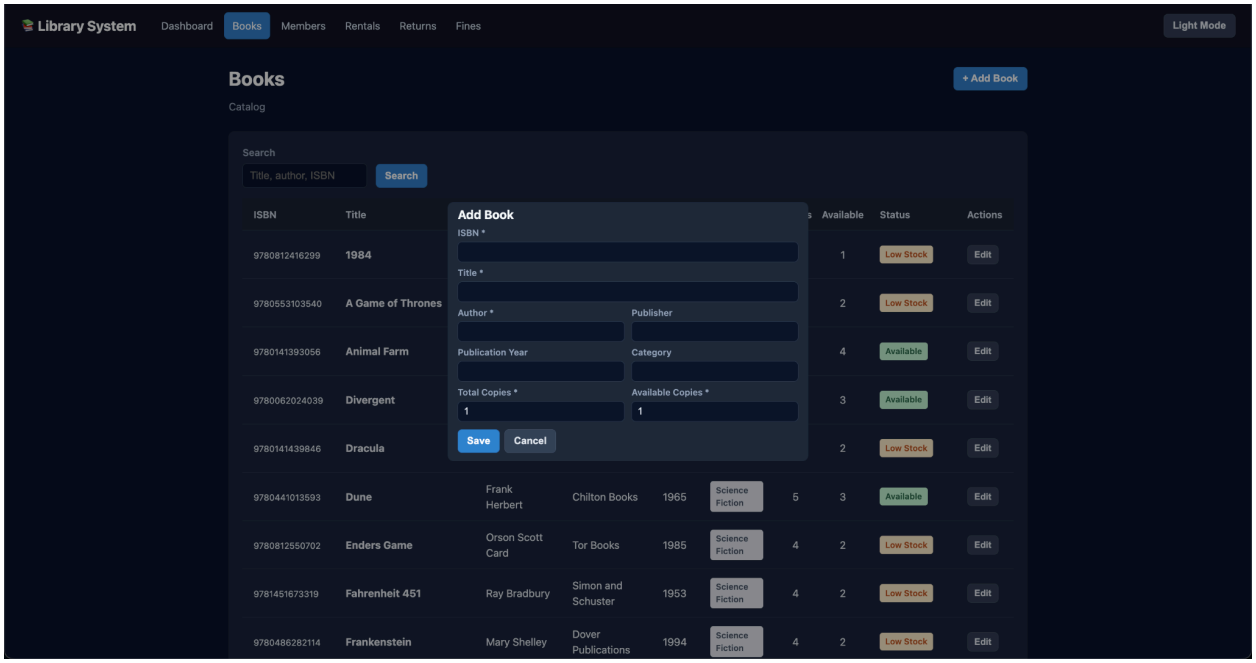
Query 3 results

book id	title	author	category	times borrowed	total copies	available copies	last borrowed date
2	Pride and Prejudice	Jane Austen	Literature	3	5	5	2026-02-05
6	Jane Eyre	Charlotte Brontë	Literature	3	3	3	2026-02-13
1	Romeo and Juliet	William Shakespeare	Literature	2	5	5	2026-02-02
3	Wuthering Heights	Emily Brontë	Literature	2	4	4	2026-02-07
5	To Kill a Mockingbird	Harper Lee	Literature	2	5	5	2026-02-10
7	The Odyssey	Homer	Literature	2	3	3	2026-02-15
8	Dracula	Bram Stoker	Literature	2	2	2	2026-02-17
10	Moby-Dick	Herman Melville	Literature	2	2	2	2026-02-20
4	The Great Gatsby	F. Scott Fitzgerald	Literature	1	4	4	2026-02-08
9	Oliver Twist	Charles Dickens	Literature	1	3	3	2026-02-18

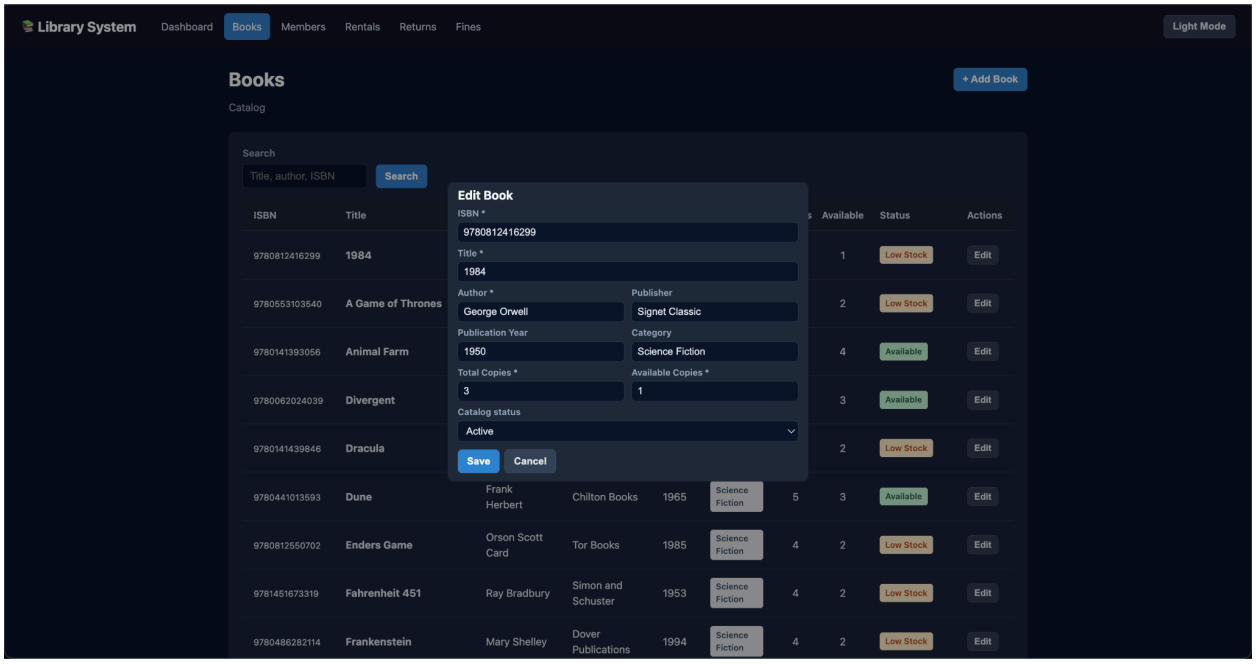
Book table



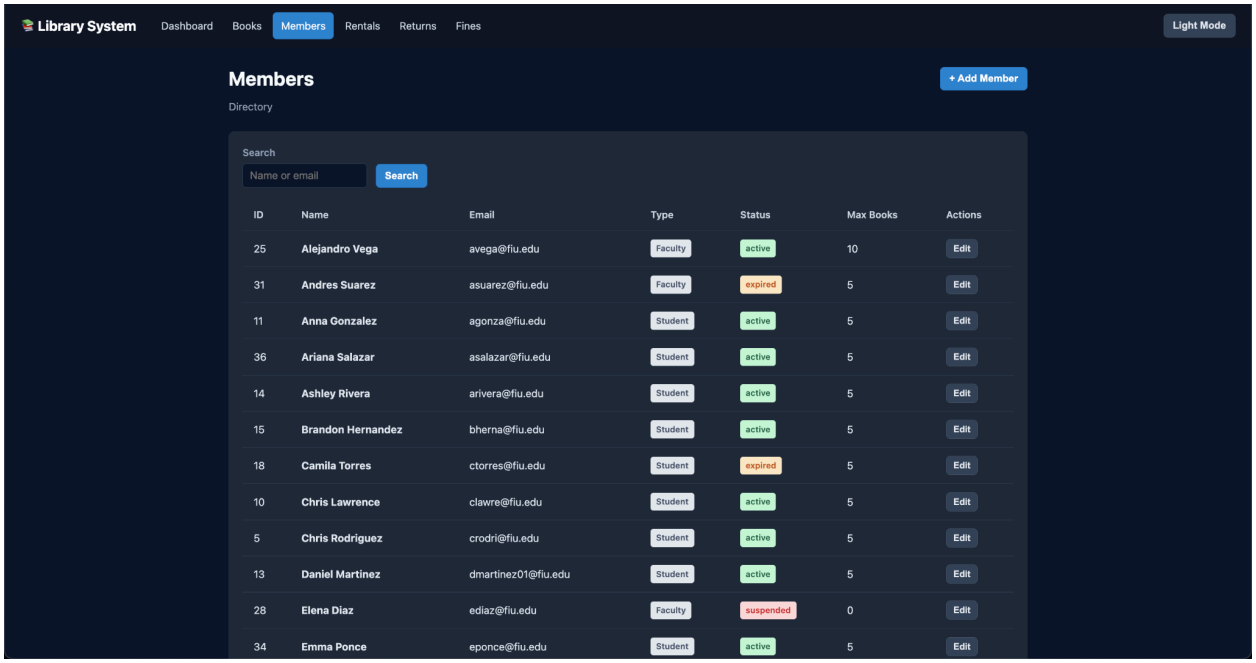
Add a new book



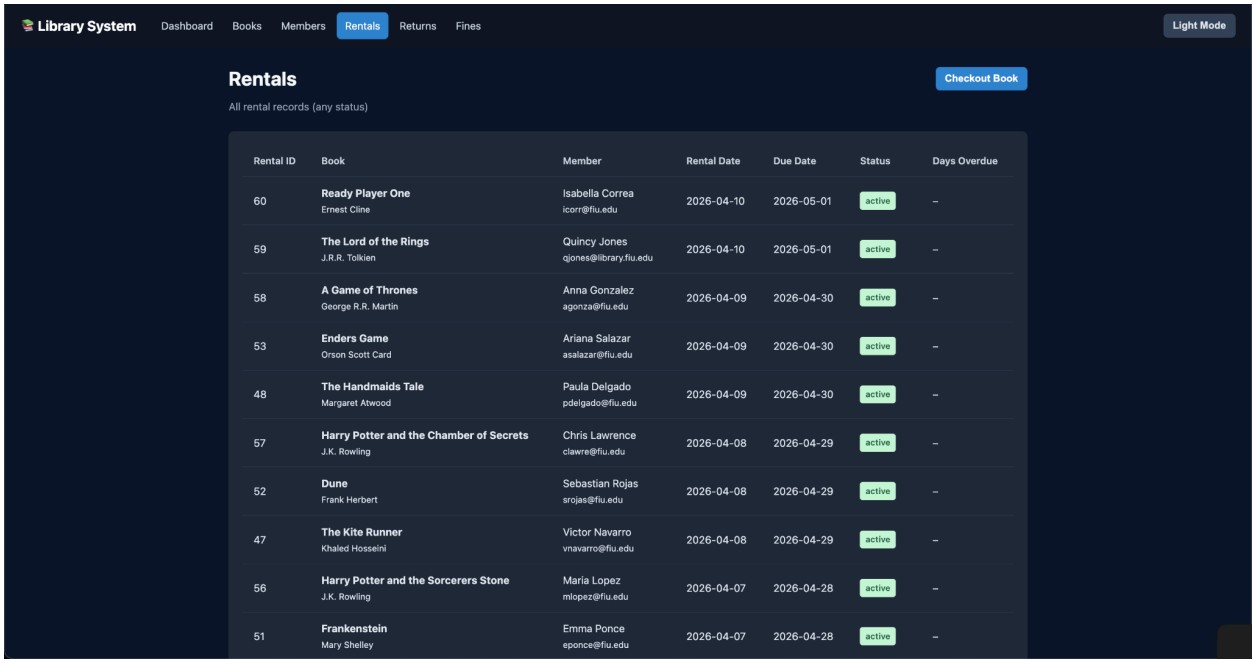
Edit an existing book



Members

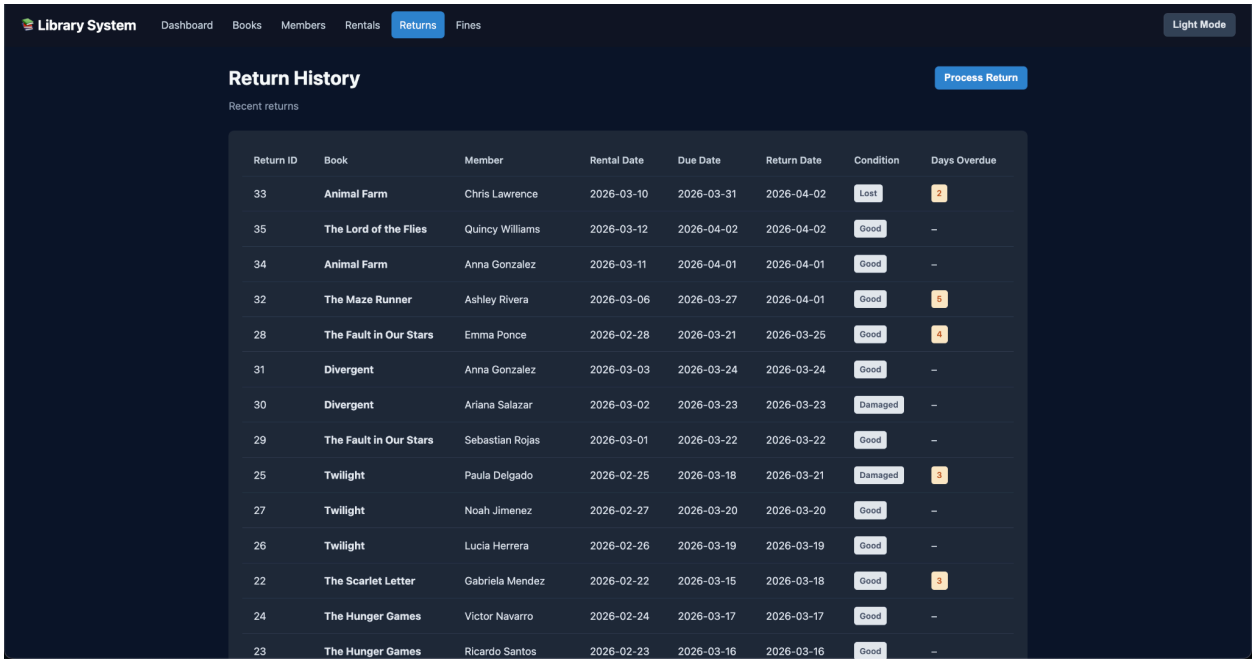


Rentals



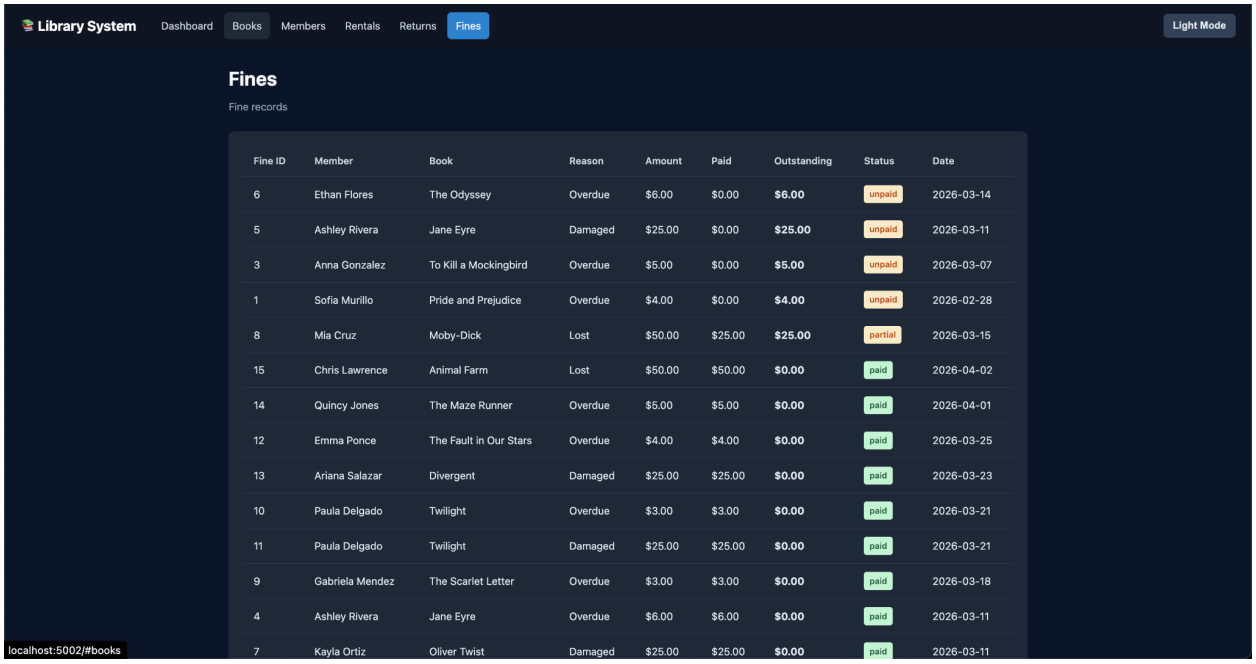
Rental ID	Book	Member	Rental Date	Due Date	Status	Days Overdue
60	**Ready Player One** Ernest Cline	Isabella Correa icorr@fiu.edu	2026-04-10	2026-05-01	active	–
59	**The Lord of the Rings** J.R.R. Tolkien	Quincy Jones qjones@library.fiu.edu	2026-04-10	2026-05-01	active	–
58	**A Game of Thrones** George R.R. Martin	Anna Gonzalez agonza@fiu.edu	2026-04-09	2026-04-30	active	–
53	**Enders Game** Orson Scott Card	Ariana Salazar asalazar@fiu.edu	2026-04-09	2026-04-30	active	–
48	**The Handmaids Tale** Margaret Atwood	Paula Delgado pdelgado@fiu.edu	2026-04-09	2026-04-30	active	–
57	**Harry Potter and the Chamber of Secrets** J.K. Rowling	Chris Lawrence clawre@fiu.edu	2026-04-08	2026-04-29	active	–
52	**Dune** Frank Herbert	Sebastian Rojas srojas@fiu.edu	2026-04-08	2026-04-29	active	–
47	**The Kite Runner** Khaled Hosseini	Victor Navarro vnavarro@fiu.edu	2026-04-08	2026-04-29	active	–
56	**Harry Potter and the Sorcerers Stone** J.K. Rowling	Maria Lopez mlopez@fiu.edu	2026-04-07	2026-04-28	active	–
51	**Frankenstein** Mary Shelley	Emma Ponce eponce@fiu.edu	2026-04-07	2026-04-28	active	–

Returns



Return ID	Book	Member	Rental Date	Due Date	Return Date	Condition	Days Overdue
33	**Animal Farm**	Chris Lawrence	2026-03-10	2026-03-31	2026-04-02	Lost	2
35	**The Lord of the Flies**	Quincy Williams	2026-03-12	2026-04-02	2026-04-02	Good	–
34	**Animal Farm**	Anna Gonzalez	2026-03-11	2026-04-01	2026-04-01	Good	–
32	**The Maze Runner**	Ashley Rivera	2026-03-06	2026-03-27	2026-04-01	Good	5
28	**The Fault in Our Stars**	Emma Ponce	2026-02-28	2026-03-21	2026-03-25	Good	4
31	**Divergent**	Anna Gonzalez	2026-03-03	2026-03-24	2026-03-24	Good	–
30	**Divergent**	Ariana Salazar	2026-03-02	2026-03-23	2026-03-23	Damaged	–
29	**The Fault in Our Stars**	Sebastian Rojas	2026-03-01	2026-03-22	2026-03-22	Good	–
25	**Twilight**	Paula Delgado	2026-02-25	2026-03-18	2026-03-21	Damaged	3
27	**Twilight**	Noah Jimenez	2026-02-27	2026-03-20	2026-03-20	Good	–
26	**Twilight**	Lucia Herrera	2026-02-26	2026-03-19	2026-03-19	Good	–
22	**The Scarlet Letter**	Gabriela Mendez	2026-02-22	2026-03-15	2026-03-18	Good	3
24	**The Hunger Games**	Victor Navarro	2026-02-24	2026-03-17	2026-03-17	Good	–
23	**The Hunger Games**	Ricardo Santos	2026-02-23	2026-03-16	2026-03-16	Good	–

Fines



Fine ID	Member	Book	Reason	Amount	Paid	Outstanding	Status	Date
6	Ethan Flores	The Odyssey	Overdue	$6.00	$0.00	**$6.00**	unpaid	2026-03-14
5	Ashley Rivera	Jane Eyre	Damaged	$25.00	$0.00	**$25.00**	unpaid	2026-03-11
3	Anna Gonzalez	To Kill a Mockingbird	Overdue	$5.00	$0.00	**$5.00**	unpaid	2026-03-07
1	Sofia Murillo	Pride and Prejudice	Overdue	$4.00	$0.00	**$4.00**	unpaid	2026-02-28
8	Mia Cruz	Moby-Dick	Lost	$50.00	$25.00	**$25.00**	partial	2026-03-15
15	Chris Lawrence	Animal Farm	Lost	$50.00	$50.00	**$0.00**	paid	2026-04-02
14	Quincy Jones	The Maze Runner	Overdue	$5.00	$5.00	**$0.00**	paid	2026-04-01
12	Emma Ponce	The Fault in Our Stars	Overdue	$4.00	$4.00	**$0.00**	paid	2026-03-25
13	Ariana Salazar	Divergent	Damaged	$25.00	$25.00	**$0.00**	paid	2026-03-23
10	Paula Delgado	Twilight	Overdue	$3.00	$3.00	**$0.00**	paid	2026-03-21
11	Paula Delgado	Twilight	Damaged	$25.00	$25.00	**$0.00**	paid	2026-03-21
9	Gabriela Mendez	The Scarlet Letter	Overdue	$3.00	$3.00	**$0.00**	paid	2026-03-18
4	Ashley Rivera	Jane Eyre	Overdue	$6.00	$6.00	**$0.00**	paid	2026-03-11
7	Kayla Ortiz	Oliver Twist	Damaged	$25.00	$25.00	**$0.00**	paid	2026-03-11